

RPi Temperature Sensor

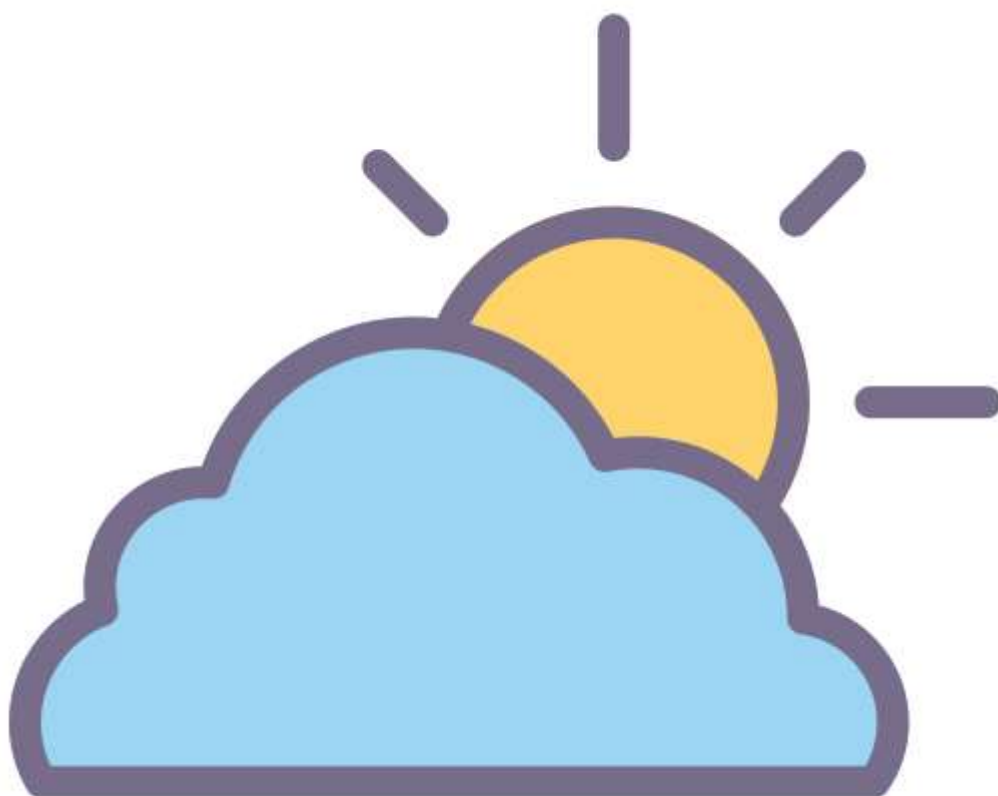


Table of Contents

Introduction.....	3
Exploration.....	4
Design.....	6
Hardware.....	6
Software.....	8
Implementation.....	10
Hardware.....	10
Software.....	13
Python Script.....	14
ISStreamer (Initial State Streamer).....	16
Results.....	20
Conclusion and Future Considerations.....	24

Introduction

In the modern world, smart technology is all around us, affecting our daily lives. It gives us comfort, accessibility and ease-of-use flexibility that was not possible before the advent of such technology. We call this the Internet of Things (IOT), where all kind of standard household items are connected to the internet, giving said objects "intelligence" due to their new-found internet connectivity. This is what we call smart objects as they communicate with other devices via the internet and relay information in more detailed, useful ways.

With the emergence of this market, these types of devices exist all over people's home these days, effectively creating a smart home. Lots of big companies are making devices for smart homes (Amazon's Alexa or Google Home for example). This does not mean that creating smart devices for your home is limited to big companies with even bigger budgets, anyone can create their own inexpensive IOT smart home.

The project I will be undertaking is an example of such, that can be created by anyone. Many different strategies exist, along with several hardware options for anyone's preference, along with the communities who thrive on creating a DIY (do it yourself) smart home, helping with the implementation of mini projects.

The IOT project decided on is a smart home temperature monitoring system. In short, I will use an IOT device, connected to a sensor to relay information onto a separate screen (such as a PC or mobile device) as a means of easily monitoring temperature from not only inside the home, but also from outside. This is the critical IOT aspect, being able to use temperature monitoring through the use of a smart device that connects to the internet, which in this case is a Raspberry PI, which is popular for many kinds of IOT projects.

This report will be divided into 5 parts. **First**, exploration, which will go into my research and inspiration, why I decided on the project and what decisions I made to start working on the project, along with thought out decisions on each aspect of the project. **Second**, design, covering the design decisions I made, from both the hardware and software side of things, including diagrams and any changes made during the project. **Third**, implementation, and how I managed to get all of the hardware and software programmatically (showing some code) connected together, forming the architecture and effectively create the smart device. **Fourth** and most importantly, the results, where I discuss how well the IOT projects functions and if things are working as expected, including the actual data from the project. **Fifth**, the conclusion, reflecting on my work, addressing any considerations or challenges and perhaps discussing future use of the IOT project, or further developments.

Exploration

An IOT project must start with some inspiration in mind, as there needs to be meaning to the project you will undertake. With that in mind, I started by looking for a project that was not only simple, but also practical for my personal use.

In the beginning, I was looking at several different mini IOT projects since as I did not have anything in mind when beginning the course. It appeared there were not too many applications of IOT I could accomplish in my household, as things such as a smart fridge are too ambitious. I then looked at a lot of simpler IOT projects, mainly aimed at sensors and monitoring, rather than mechanical devices that act automatically or with user input.

Knowing this, I realized that a Raspberry Pi would be a good option as the Raspberry Pi has enough power to do some relatively advanced monitoring and is best for observation-related IOT projects. Examples include projects like a burglar monitor, using motion sensors when not at home to detect for break-ins. Another would be turning any regular device into a smart fridge, being able to now monitor and set temperatures on your fridge from a distance. Additionally I already have some familiarity with the Python programming language, which is the primary language for the majority of Raspberry Pi projects.

The Arduino, an alternative IOT device, is more suited for mechanical IOT projects, where moving parts are required. Such an example would include a smart garden, where water needs to be administered to plants and the air circulation must be regulated with fans. Another would be a pet food dispenser, where food and water would be poured into bowls based on weight sensor data.

With this I settled on a temperature sensor which, as described earlier, will simply monitor temperature in a given room in my home, where I can observe the readings over the internet using an IOT web service. Since the key aspect is constant observation of data, the Pi is the final choice I made. I additionally chose it

I chose this project due to its simplicity and practicality. Since I have little prior experiences with IOT devices, and no experience undertaking an IOT project, I realized a more simple IOT project would be necessary if I am to create a functioning, successful smart implementation in my home regarding the time and experience I have.

This project is practical for me because I do not currently have any indoor temperature monitor, nor do I have any “advanced” (only a basic, analogue radiator and no air conditioner or fans) ways of cooling or heating the house. In my case, having a smart temperature monitor can be

28/04/21

considered advanced relative to how I regulate temperature in the house, and since a sensor for a smart device can be relatively accurate and fast, means that I can act accordingly and with good time.

The process is fairly simple, I realized implementation takes priority over the design, simply because there are not too many physical components involved, which also makes the software structure fairly simple. Implementation will be the larger focus, since I believe the challenge for this project is being able to display the readings on a server, and being able to see the necessary data via the internet.

Many people have already done a project similar to this one as monitoring temperature at home seems like a practical, no hassle choice for many home owners. There already exist many code repositories, Python libraries and physical sensor solutions for this type of project on the web.

Concluding this chapter, my project is not original, but I consider it a good starting point for learning and implementing an IOT project.

Design

Design is an important part of any IOT project. In order for any project to be implemented properly, it must first be designed accordingly.

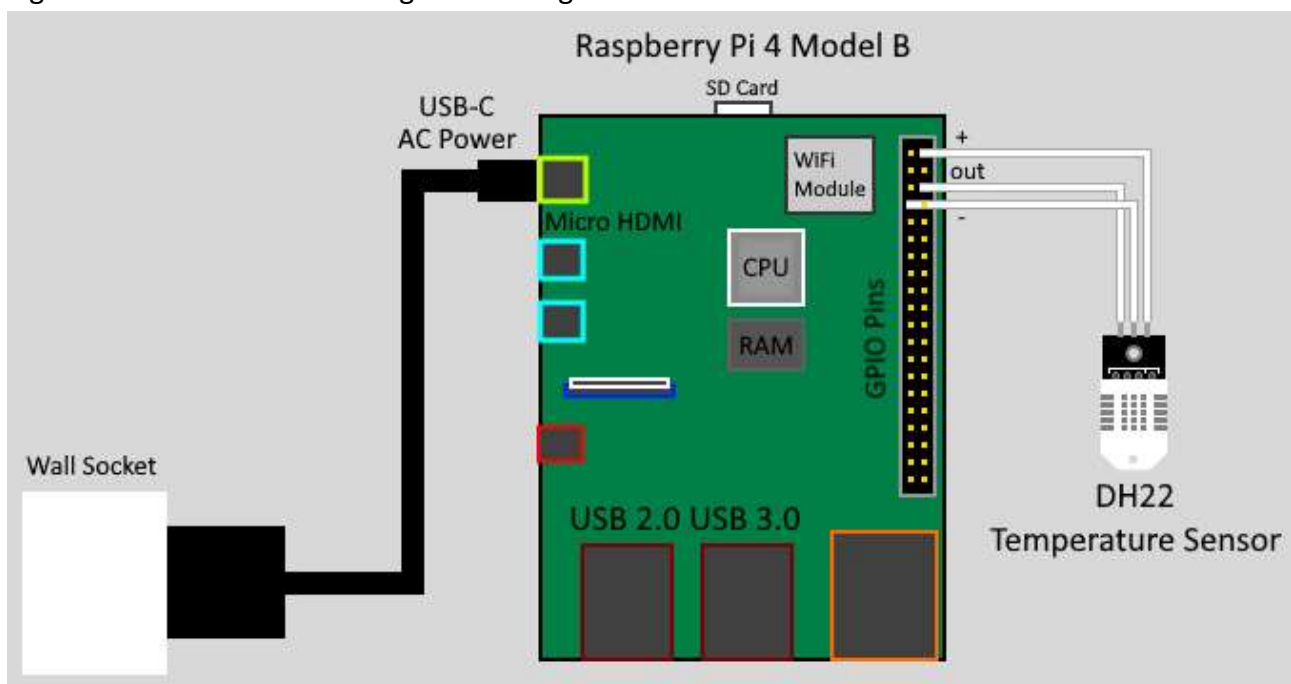
For my project, I aimed to make the design as straightforward and minimal as possible.

For this section, I have decided to split up the design of my project into 2 parts. The first part is hardware, and how I designed the project to look physically. The second part is software, and how I designed the architecture of the project.

Hardware

When discussing the design in regards to hardware, the primary focus is on how the hardware is set up in the environment. Despite this, what can be attributed to the design is that the size of the components chosen for the project allow the setup to be more flexible regarding placement and the room taken up.

Figure 1 – Basic hardware design and configuration:



(Parts indicated with text are the ones relevant to this project)

Figure 1 depicts the basic hardware design for this project. What is clear about this illustration is the demonstration of the few components needed to undertake this projects, at least from a hardware perspective.

There are 3 main components to this project while the IOT setup is active.

First, the Raspberry Pi. The microcomputer needed to send all the data to the server, where it will eventually be visible to the end user. All of the programming and data receiving and sending are done on this machine. Due to the Pi's versatility, many devices and sensors can be connected through USB (Universal Serial Bus) ports or the GPIO (General-purpose input/output) pins.

Second, the DHT22 temperature sensor. This is the edge computing IOT sensor that captures both the current temperature and humidity within the proximity of the sensor. This data is sent directly to the Raspberry Pi through the connected wires. This data is only interpreted by the Pi, but not modified until it is sent up to the server.

Third, the wall socket, which gives power to both the Pi and DHT22 via a AC powered USB-C connector. The Pi needs a sufficient amount of power to be able to operate (At least 3 Amps)², thus plugging a USB-C connector into a USB3.0 port on a PC for example, will not provide sufficient power for both components, let alone the Pi.

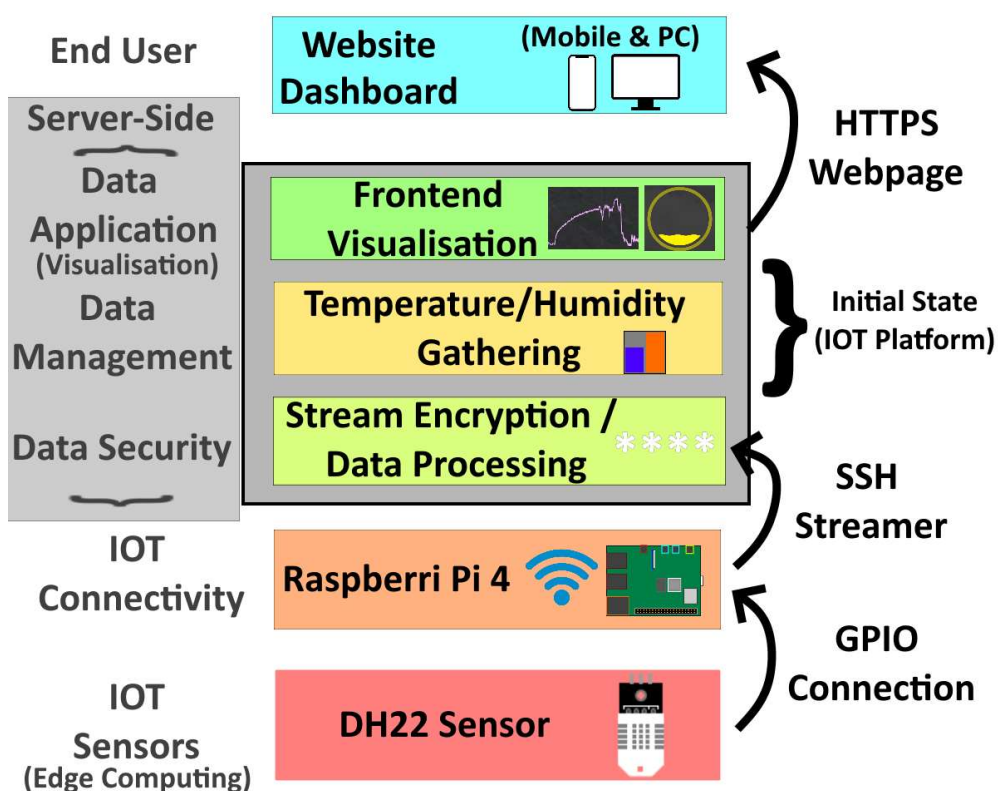
Because of the non-complex nature of the setup, improvements can still be made in the future. For example, a custom case for the Raspberry Pi can be constructed specifically for this project (one that fits the Pi and sensor comfortably), which not only improves aesthetics, but will also be more convenient for the sensor connected to the Pi.

As this project is essentially a prototype, all of the components are 'in the open', on the workstation. No complicated hardware setup was put together at this stage in the project. This will be expanded further in the implementation section.

Software

Not only must a hardware setup be realized through design, but so must the software side of the project. In this case, the general architecture, and software process is considered, as it takes up the majority of the project's design from a more back-end perspective. As this is an IOT project, certain layers are to be described, such as the server side of the project, down to the edge computing ('I' (internet) and 'T' (things) in IOT respectively).

Figure 2 – Basic software design architecture, process and layers:



(Left: Basic terminology, Center: Core components/features, Right: software process)

Figure 2 depicts the software structure for the entire IOT project. There are 3 main sections of this architecture.

First, the software of the hardware. The DHT22, being an edge computing sensor, has embedded software that sends readable data to another device. The Raspberry Pi, being the IOT connectivity device, uses the Python programming language to interpret this data into readable information, sent via the GPIO connections discussed earlier. The Pi, sends this data via WiFi connection (with SSH enabled for a secure data stream) to the internet.

28/04/21

Second, the server side of the IOT project. When the data is sent from the Pi, it is encrypted via a public key. This data is being encrypted processed by the Initial State back end. The 2 data captures, temperature and humidity, and then managed by the platform once streamed. Lastly, the data is then 'applied' via simple visualization, such as a time series.

Lastly, the end user. Once the process is complete, the end user can view the streamed data in an easy-to-understand website dashboard, viewable and configurable from both a desktop and mobile device.

Using different devices/sensors or IOT platforms may alter some more specific details of this general design, but when creating a temperature monitoring IOT project, many of these fundamental components remain the same.

Implementation

Design is only how a project is planned via specification, but implementing any design is another entire aspect. This is arguably the most important section, as it showcases how this IOT project came to life.

Like with the design chapter of this report, I will split this section up into the 2 respective parts. First is hardware, and how everything was constructed from a physical standpoint to create a functioning IOT device. The second is again software, and how the code and IOT environment was implemented via programming to function correctly.

Hardware

The hardware implementation side of things happens to be rather simple. When functioning, there are only a few components to worry about.

Figure 3 – Hardware configuration in action:

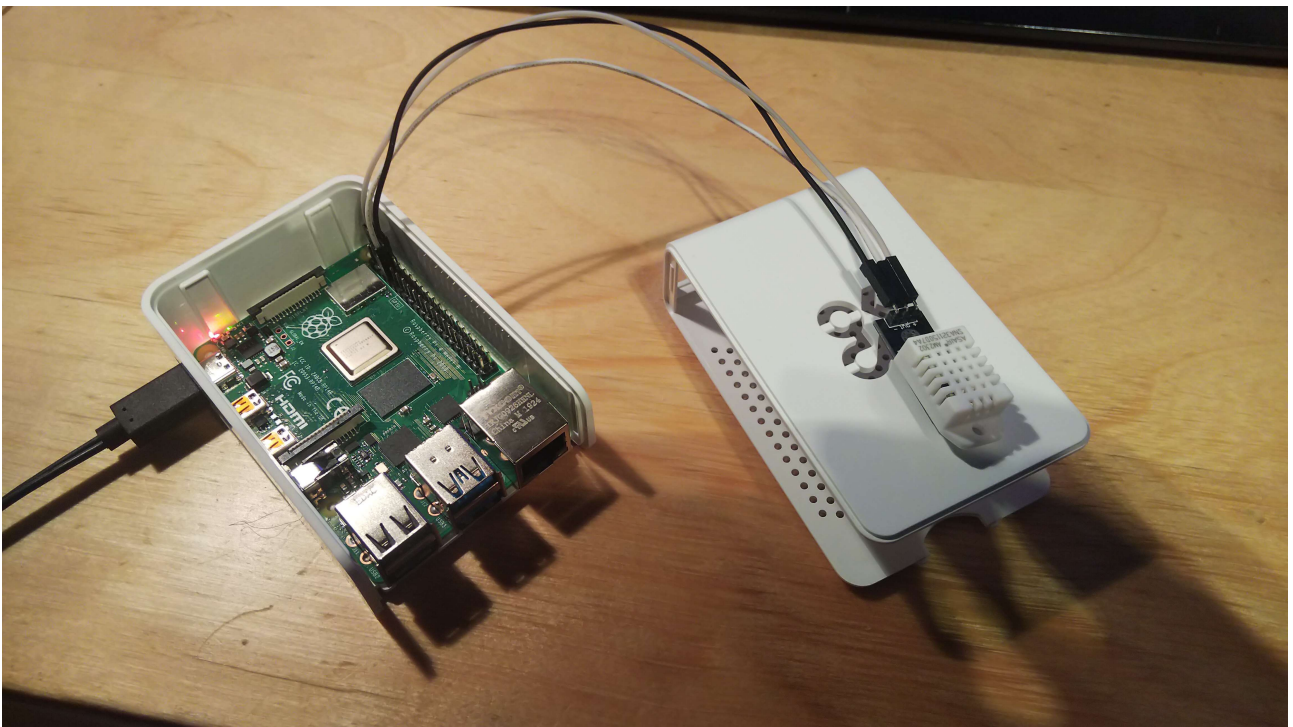


Figure 3 depicts the components necessary for the working IOT temperature monitor. In short, we have the black AC adapter connected to the wall via a power socket and USB-C connection (left). Additionally, the DHT22 is connected to the Pi via pin connectors (center-right). While there is an indication of the Pi running (Red and green LEDs), there is not any indicator for the sensor.

Setting up the hardware is rather simple.

First, an SD card is inserted into the back of the Pi, containing an OS compatible with the device. Second, the Pi is plugged into the mains. The reason this comes second is because it powers on as soon as it plugged in, so the OS must be ready as this occurs.

Lastly the sensor is wired to the Pi. This step can occur regardless if the Pi is powered on or not. The important aspect, is making sure it is wired correctly. As shown in **Figure 1**, The rightmost 'power' connection (indicated with '+') must be connected to the 5V pin on the Pi. The leftmost ground pin (indicated with '-') must be connected to ground pin 6. The center pin (indicated with 'out') can be connected to any of the GPIO pins, in this case, it will be connected to GPIO 4.

Other hardware is also necessary when setting the project up. In **Figure 1**, other components necessary on the Pi are highlighted. The USB ports are necessary for keyboard and mouse usage, as certain things on the Raspberry Pi OS (operating system) must be set up for the sensors to work. This will be discussed in more detail in the software section below. Other components like the micro HDMI ports are needed to see the screen while operating the Pi. Finally, an SD card is required to install the OS, and thus operate the Pi from any level.

Figure 4 – Hardware setup needed to utilize the Raspberry Pi:



Figure 4 depicts the utilization of other hardware (micro HDMI to a monitor, keyboard and mouse plugged into the USB3.0 ports) to set up the project on the Pi itself.

28/04/21

Other non-intractable components on the Pi are also fundamental, such as the CPU (central processing unit), necessary for processing the sensor data and running the code, and the WiFi/Bluetooth module, to be able to upstream the data to an internet server.

Software

The software side of the implementation process was also meant to be simple. The plan was to set up and configure the Pi for use as a mini PC, so all of the software implementation can happen on it. This includes testing the data stream, and writing the code inside a Python IDE (Integrated Development Environment).

With that in mind, the desktop-esque Raspberry Pi OS was loaded onto an SD card, then automatically configured once the Pi is turned on, as it recognizes the data on the SD card.

Figure 5 – Raspberry Pi OS desktop:

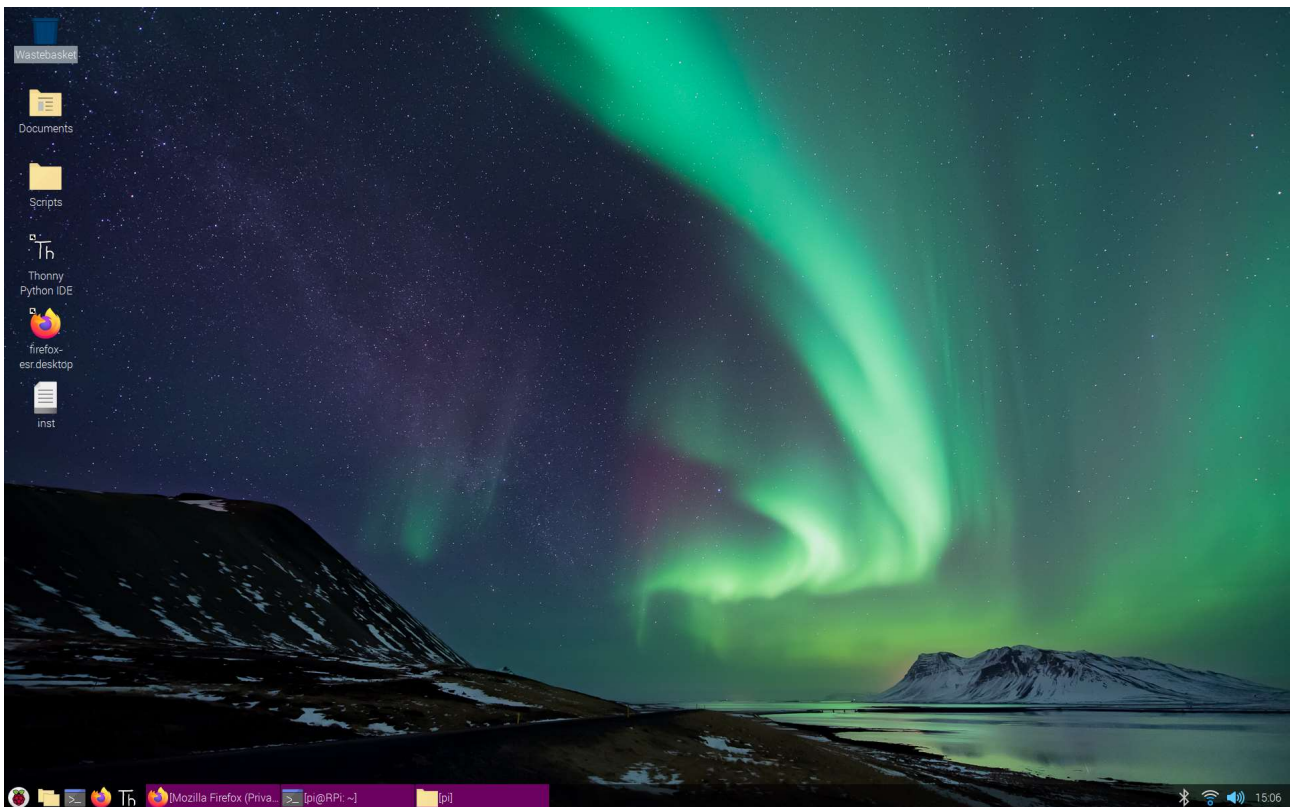
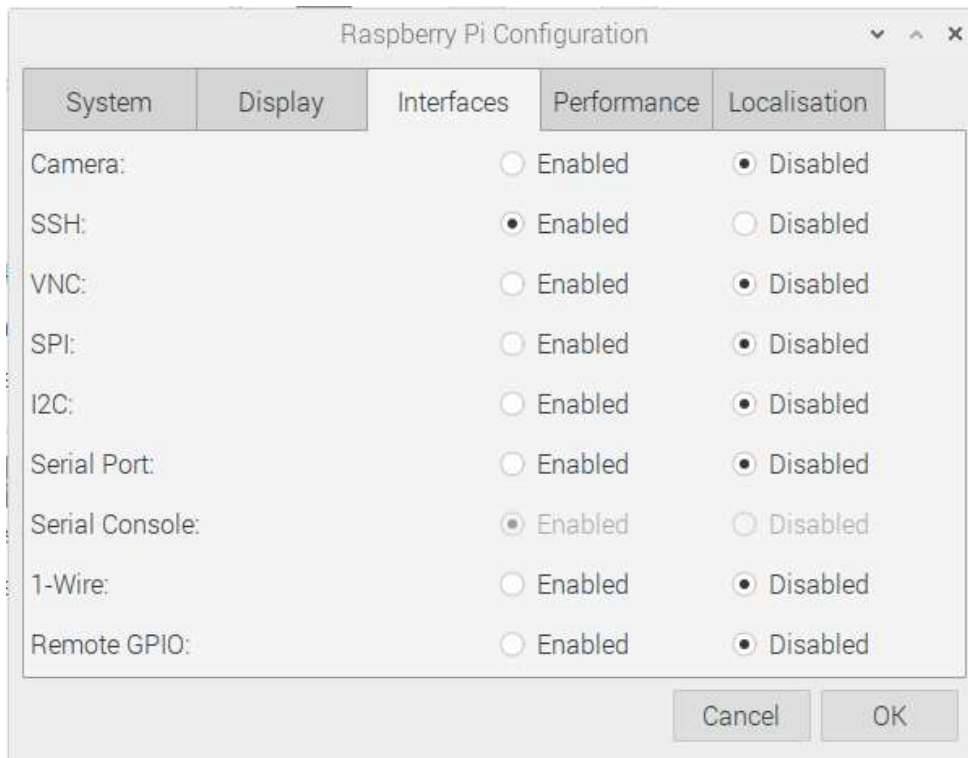


Figure 5 depicts how the OS's desktop appears, complete with a start menu, taskbar and desktop icons. This essentially makes up the general environment for complete software implementation.

As a prerequisite to the implementation, certain processes such as WiFi, must be enabled to connect to the internet. SSH (Secure Shell) must also be enabled, allowing potential data to be streamed securely, as that data will now be encrypted.

Figure 6 – Enabling SSH through the Raspberry Pi Configuration menu:



Now that all of the essential early steps are complete, we can move onto the Python scripting.

Python Script

Getting the project up and running from a programming perspective required just one Python script, as it would handle most of the back end work.

Before writing any script, a few Python libraries must be installed for everything to function properly.

First, the Adafruit CircuitPython_DHT library. This is the library that allows the use of the DHT22 (as well as other sensors) that is being used for this project. It 'enables' the ability to read temperature/humidity from the sensor, onto a Raspberry Pi³. This is the most important library for the code to function.

Second, the ISStreamer library. This package makes the process of streaming data to the Initial State platform simple⁴. this will be expanded further in the ISStreamer section.

Third, the time library, a module apart of the Python standard library, which will allow the control of intervals between data readings⁵.

28/04/21

Last, the board library. This allows us to tell the script which pins are connected with the sensor, allowing the reading to occur.

Once all of these libraries are installed and imported, the Python code can now be written.

Figure 7 – The Python script used for the project

```
import adafruit_dht
from ISStreamer.Streamer import Streamer
import time
import board

# ----- User Settings -----
SENSOR_LOCATION_NAME = "Josef's Office"
BUCKET_NAME = ":partly_sunny: Room/Outside Temperature"
BUCKET_KEY = "P5GRHDKAJMG3"
ACCESS_KEY = "ist_ac_m5Fd0QMvxbAVV_qCX0DwLBEM6U-n4"
SECONDS_BETWEEN_READS = 2
#METRIC_UNITS = True
# -----

dhtSensor = adafruit_dht.DHT22(board.D4)
streamer = Streamer(bucket_name=BUCKET_NAME, bucket_key=BUCKET_KEY, access_key=ACCESS_KEY)

while True:
    try:
        humidity = dhtSensor.humidity
        temp_c = dhtSensor.temperature
        print("Temperature: " + str(temp_c) + "C " + "Humidity: " + str(humidity) + "%")
    except RuntimeError:
        print("RuntimeError, trying again...")
        continue

    streamer.log(SENSOR_LOCATION_NAME + " Temperature(C)", temp_c)
    humidity = format(humidity, ".2f")
    streamer.log(SENSOR_LOCATION_NAME + " Humidity(%)", humidity)
    streamer.flush()
    time.sleep(SECONDS_BETWEEN_READS)
```

Figure 7 depicts the code for this project in its entirety. It is a modified version of Elizabeth Adam's *tempsensor.py*⁶. The goal was to make the code more simple, changing and removing unnecessary components.

The code does everything necessary for the whole project to work, including retrieving the data, displaying it on the Pi, then streaming it to the Initial State servers.

In the first 4 lines, all the necessary libraries are imported.

Under the 'User Settings' comment, all of the necessary server-side user configurations are present. `SENSOR_LOCATION_NAME` and `BUCKET_NAME` are both for the IOT dashboard that will

be shown in the *Results* section. `BUCKET_KEY` is key that makes sure the data goes to the right place on the dashboard. `ACCESS_KEY` is the stream key, essentially an encryption key making sure the data is streamed without malicious interference. `SECONDS_BETWEEN_READS` (modified from the original `MINUTES_BETWEEN_READS`, for more precise data gathering) indicates the intervals between reads, i.e. how often the sensor readings will be output.

The *dhtSensor* variable declares which sensor we are using (adafruit_dht.DHT22 for the DHT22 sensor, *board.D4* as the GPIO 4 pin is being used for the data stream).

The *streamer* variable sets up the contents for the dashboard, for which the *access_key* from before is needed.

A while loop is created for the next section as the data readings from the sensor are not 100% reliable, the data may sometimes come back as NULL. These NULL readings will just be returned as `RuntimeError` and the stream continues regardless. The code under *try* declares variables for temperature and humidity and outputs these values into the command line, just to make sure the reading are functioning properly.

The *streamer.log* are essentially the methods that send the 2 types of data to the server. The *flush* method flushes the data by clearing it from the buffer, then sending that data up⁷. Finally, the *sleep* function stops the script for a desired amount of seconds, then resumes it from the beginning of the loop.

From the original code, some of the data reading format was deleted, as it caused errors when executing the script. Additionally, the `METRIC_UNITS` variable was removed, as there was no intention on using Fahrenheit temperature readings.

Now that the code has been reviewed, the data stream can commence.

ISStreamer (Initial State Streamer)

This is the core of this project and what makes it an IOT one. There are many IOT dashboards and services one can use for their project, and in this case Initial State was chosen as it offers all the things necessary for this project to run successfully and is relatively easy to set up.

In order to start using Initial State, the ISStreamer must be installed and configured.

Figure 8 – Installing the ISStreamer through the Raspberry Pi terminal

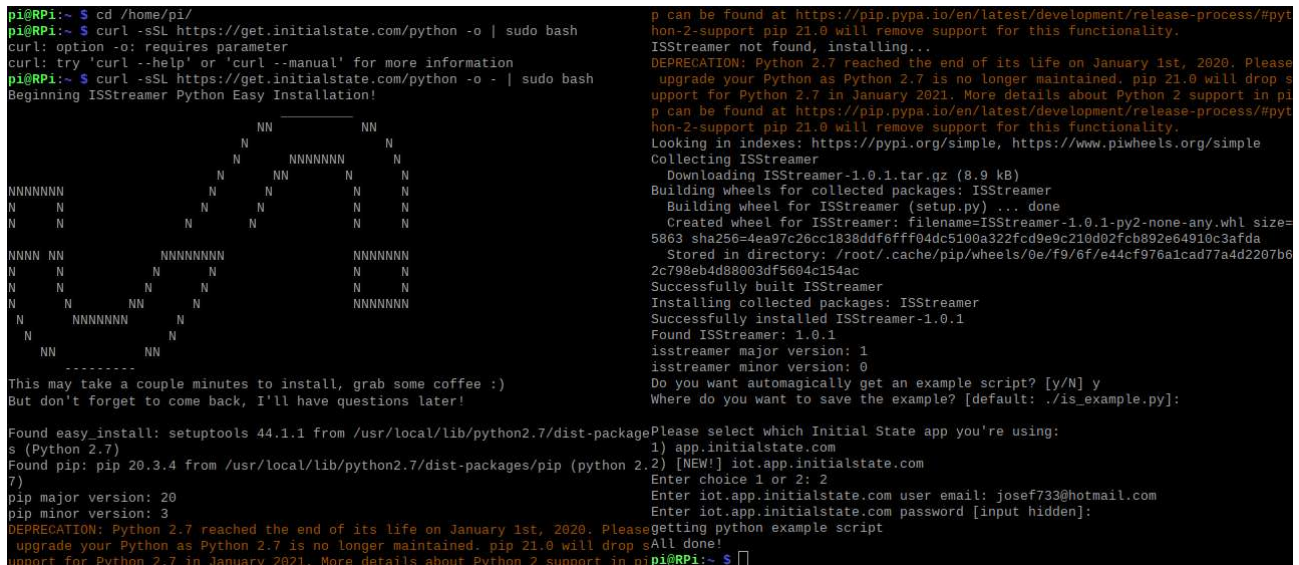


Figure 8 depicts the simple and mostly automatic install of the streamer, so Python code can be interpreted and sent up to the server.

While the process is mostly automatic, the user must still chose what version of the site shall be used. In this case the 'New' site will be used (option 2). The user must also provide the credentials of the account that was used to sign up to iot.app.initialstate.com.

Once the correct information is provided and the modules are installed, example code is provided to test if everything functions and is ready to be used for the IOT project.

28/04/21

Figure 9 – The example code retrieved during ISStreamer installation

```
GNU nano 3.2 is_example.py
#####
# This is a custom 'getting started' script, made with care for josef733@hotmail.com.
# If you have any questions, please email us! support@initialstate.com
#####

# Import the ISStreamer module
from ISStreamer.Streamer import Streamer
# Import time for delays
import time

# Streamer constructor, this will create a bucket called Python Stream Example
# you'll be able to see this name in your list of logs on initialstate.com
# your access key is a secret and is specific to you, don't share it!
streamer = Streamer(bucket_name="Python Stream Example", bucket_key="python_example", access_key="ist_ac_m5Fd0QMvxbAVV_qCX0DwLBEM6U-n4")

# example data logging
streamer.log("My Messages", "Stream Starting")
for num in range(1, 20):
    time.sleep(0.1)
    streamer.log("My Numbers", num)
    if num%2 == 0:
        streamer.log("My Booleans", False)
    else:
        streamer.log("My Booleans", True)
    if num%3 == 0:
        streamer.log("My Events", "pop")
    if num%10 == 0:
        streamer.log("My Messages", "Stream Half Done")
streamer.log("My Messages", "Stream Done")

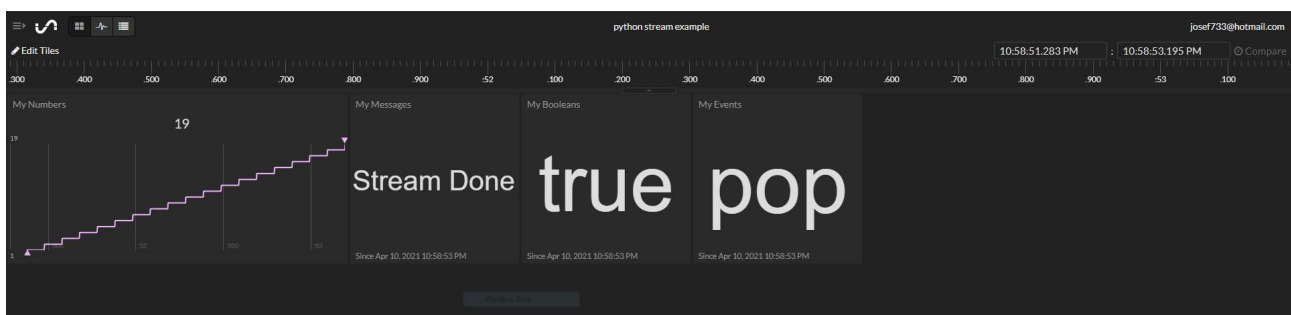
## This is just an example, try something of your own!
## ideas:
## - solve world hunger, one bug fix at a time
## - create the worlds first widget
## - build an army of bug-free robot kittens

# Once you're finished, close the stream to properly dispose
streamer.close()
```

Figure 9 demonstrates the example code for the streamer. Its functionality is very simple. Like with the actual script that will be used, the streamer variable is here and has the same bucket_name, bucket_key and access_key variables, the access_key being the same as in the actual script since it is account-based. The *log* method is used to stream messages, as well as values in a simple for loop, where conditional statements indicate what is being streamed to the dashboard. One difference is that the *close()* method is being used, as this is not a continuous stream.

When this code is executed, the result can be viewed in a test stream on the Initial State dashboard.

Figure 10 – The test stream, as seen from the Initial State dashboard.



28/04/21

Once on the dashboard, it can be seen that all the conditionals and log methods used result in this dashboard, with several different panels since many things were logged separate.

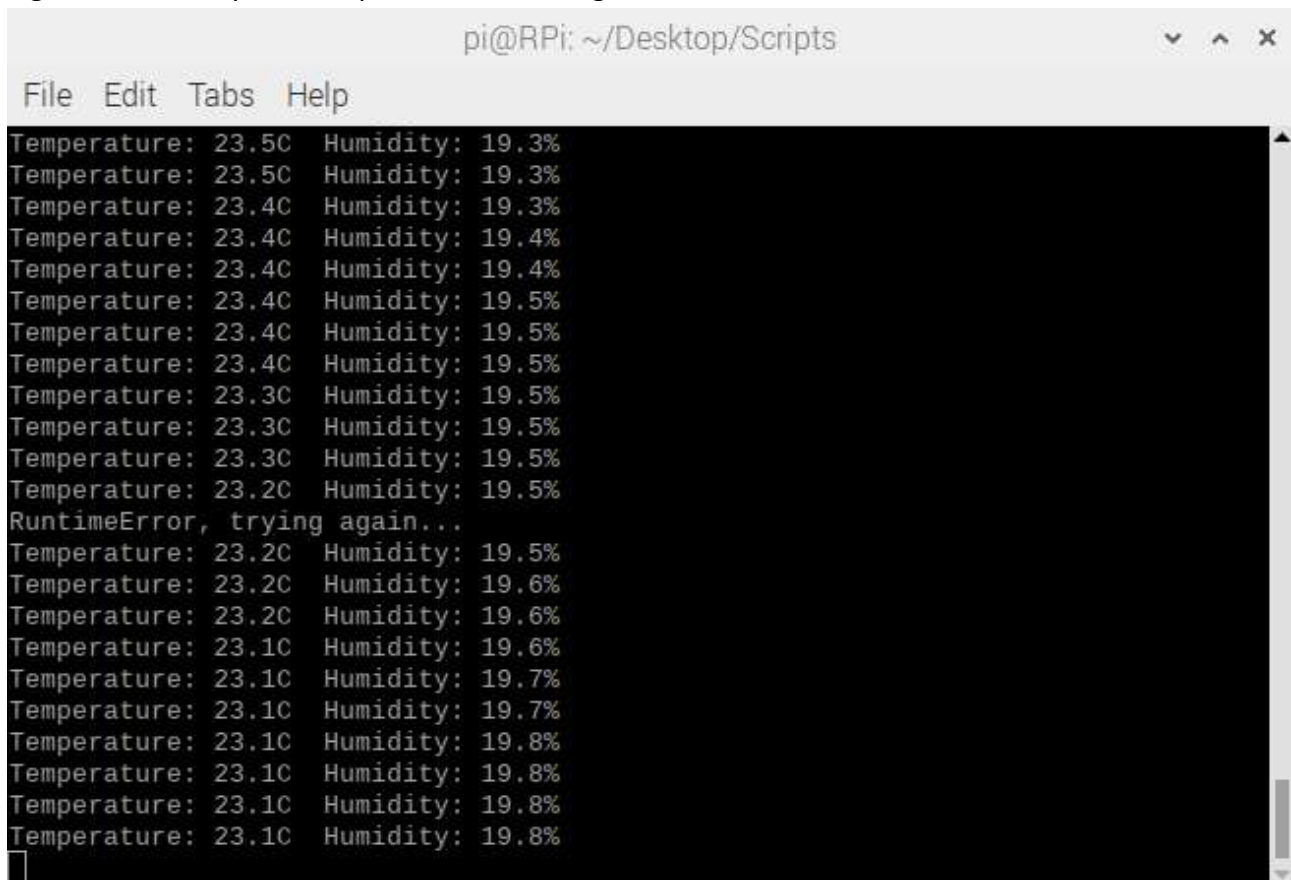
With both the hardware and software integration in place, it is time to see the project's results.

Results

For this section of the report, both the internet side, and the offline (in case internet connectivity malfunctions) side of things will be tested and made sure they are all in working order.

First, the code will be tested with the sensor wired correctly.

Figure 11 – The Python script in action, through the RPi terminal



```
pi@RPi: ~/Desktop/Scripts
File Edit Tabs Help
Temperature: 23.5C Humidity: 19.3%
Temperature: 23.5C Humidity: 19.3%
Temperature: 23.4C Humidity: 19.3%
Temperature: 23.4C Humidity: 19.4%
Temperature: 23.4C Humidity: 19.4%
Temperature: 23.4C Humidity: 19.5%
Temperature: 23.4C Humidity: 19.5%
Temperature: 23.4C Humidity: 19.5%
Temperature: 23.3C Humidity: 19.5%
Temperature: 23.3C Humidity: 19.5%
Temperature: 23.3C Humidity: 19.5%
Temperature: 23.2C Humidity: 19.5%
RuntimeError, trying again...
Temperature: 23.2C Humidity: 19.5%
Temperature: 23.2C Humidity: 19.6%
Temperature: 23.2C Humidity: 19.6%
Temperature: 23.1C Humidity: 19.6%
Temperature: 23.1C Humidity: 19.7%
Temperature: 23.1C Humidity: 19.7%
Temperature: 23.1C Humidity: 19.8%
Temperature: 23.1C Humidity: 19.8%
Temperature: 23.1C Humidity: 19.8%
Temperature: 23.1C Humidity: 19.8%
```

Figure 11 depicts the Python script running. It has been written to display temperature in degrees Celsius and humidity as a percentage. Every 4-5 seconds (user configurable) these 2 data types are displayed inside the Raspberry Pi terminal. Occasionally, a RuntimeError (as described earlier) may occur due to the readings/sensor not being 100% reliable, but the script continues regardless, looking for the next reading. It appears to be functioning correctly as the temperature received appear reasonable for indoor areas.

As this code seems to be running correctly, this data should now be streaming to the Initial State server, and to the project dashboard respectively.

28/04/21

Logging into the dashboard, the bucket created for this project is visible and can now be accessed.

Figure 12 – The default dashboard for the project



Figure 12 depicts accessing the bucket for the first time. The title for the bucket, as well as the 2 data streams, comes from the script written earlier. These can be changed on the dashboard as well. Since all of the temperature testing came from the office, the location is dubbed 'Josef's Office'. Initially, the 2 streams visible appear on a time series, ranging from the start of the reading until the present, this can be adjusted to fit the user's needs.

This particular reading was taken over a 24 hour period, and is clear that temperature variations are present. Generally when the PC is in use and ventilation in the room is limited, the temperatures are high, but when ventilation in the room is good (I.E. windows open) and the PC is in limited or no use, the temperature falls. Humidity remains generally low, as temperatures this time of the year remain low, especially during the night.

Now that a sizable reading has been taken, and the dashboard viewed, it can now be customized to the user's liking, making it more readable and more convenient to use.

28/04/21

Figure 13 – A customized dashboard

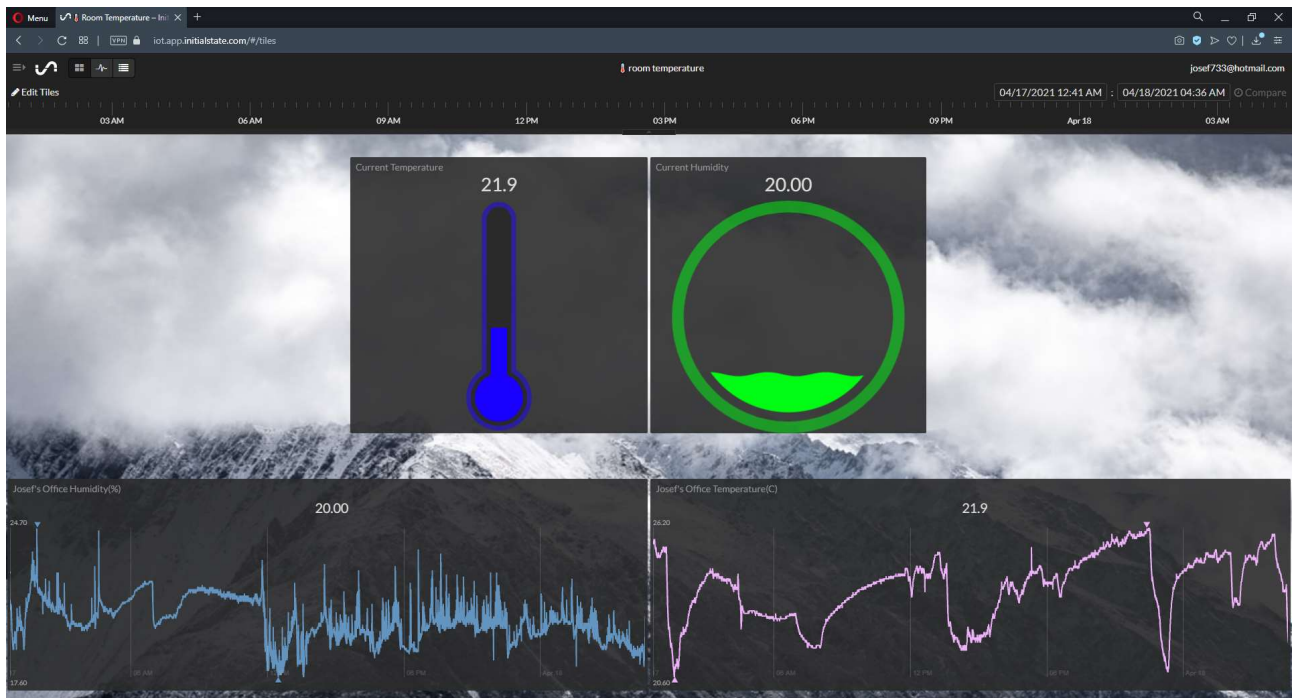


Figure 13 depicts a customized dashboard as a means of enhancing convenience.

All of the data streams can be moved around the canvas, and new widgets can be added to display the same data in a more readable way. In this instance, the time series streams were moved to the bottom as they were not as useful for general purpose. In its place, a widget for temperature and sensor are set up instead. On the left is the temperature reading, and on the right is the humidity reading. These widgets can be colour coded dynamically as well. For instance, you can have the colour of the thermometer change depending on the current temperature. In this case, the widget turns blue if the temperature falls below 22, and turns red if it rises above 25.

All of this is also viewable through a smartphone, meaning one can easily read all of this data while not at home.

28/04/21

Figure 14 – The dashboard viewed on a smartphone

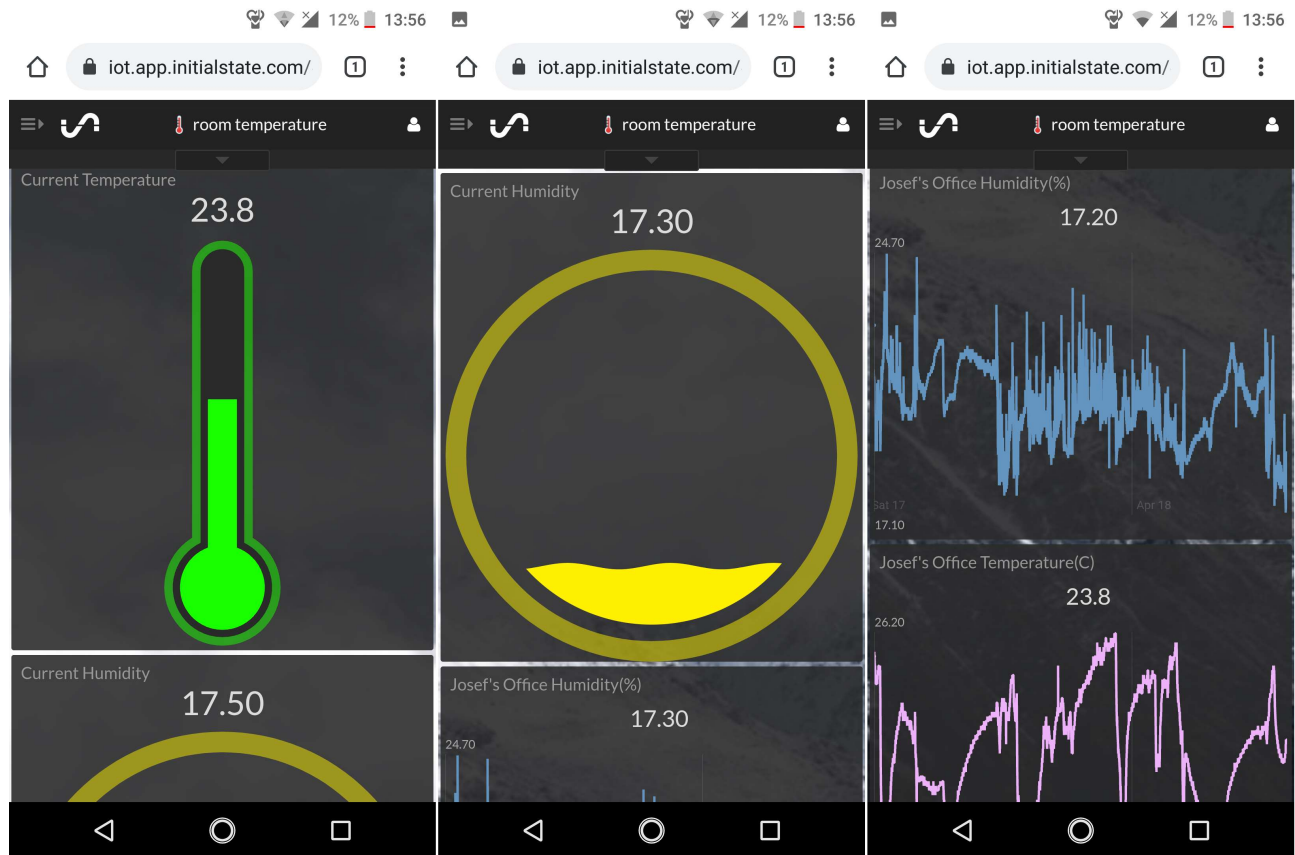


Figure 14 depicts several screenshots from an Android device of the same dashboard seen in **Figure 13**. As the Initial State dashboard is essentially a web application, no app on a smartphone is required to view this data, only a web browser with the correct URL is necessary. Everything appears to work perfectly on mobile and all the data is very clear and easy to read.

The results from this project seem quite convincing and successful. All of the data required was read successfully and uploaded to the Initial State dashboard, to be viewed in an easy, convenient way, which is a significant part of the IOT experience, and this project.

Conclusion and Future Considerations

To conclude, the project was fully realized and works as intended. The design incorporated, prior to implementing the project was fully realized at a functional level. All of the planned hardware and features from the original proposal, including notifications for weather changes, and smartphone support, are all incorporated into this project. The IOT solution and services chosen function at a more-than satisfactory level. Since this was a rather simple project, aim was to fulfill the solution to the fullest, which was what occurred during the implementation of it.

Some considerations could be made for the future, mainly pertaining to hardware design and functionality. For the design, the plan in the future is to contain all the hardware in a small package, conveniently placed on the wall, out of obstruction. A specific future hardware feature, is to include a physical LED, letting the user know what the rough temperature is without a screen, in case they are nearby. While this is already in the software solution, including relaying this information by both text and email to the user, having a physical LED would be a beneficial extra.

Bibliography

- 1: Icon Box, Sunny weather Icon,
- 2: Raspberry Pi Foundation, Power Supply,
- 3: MicroPython & CircuitPython contributors, Adafruit CircuitPython API Reference, 2021
- 4: David Sulpy, ISStreamer 1.0.1, 2019, <https://pypi.org/project/ISStreamer/>
- 5: Python Software Foundation, time — Time access and conversions, 2021
- 6: Elizabeth Adam, tempsensor.py, 2021
- 7: Elizabeth Adams, Raspberry Pi Data Logger: Raspberry Pi and Initial State, 2020